

Big Data Programming

Files and Exceptions

Chapter Overview

메모리의 값을 파일에 남기는 과정

파일의 이름과 내용을 구분한다

현재 작업 폴더와 경로를 먼저 확인한다. 파일을 연다는 것은 내용을 곧바로 읽는다는 뜻이 아니며, 실제 데이터는 read()와 같은 메서드로 얻는다.

읽는 방법과 저장하는 방법을 연결한다

문자열 전체, 한 줄, 줄 목록의 차이를 실행 결과로 비교한다. 파일에서 읽은 문자열을 나누고 숫자로 변환한 뒤 다시 파일에 쓴다.

작은 입력으로 결과를 검증한다

전화번호 파일과 근무 시간 자료를 읽고, 텍스트와 이미지 파일을 복사한다. 쓰기 모드와 출력 파일 이름을 확인하여 실습 원본을 지키는 습관을 익힌다.

Files

프로그램을 다시 실행해도 값을 사용하려면

변수만으로는 다음 실행까지 보관하지 못한다

리스트와 딕셔너리는 실행 중인 Python의 메모리에 있다. 커널을 다시 시작하면 직접 저장하지 않은 변수 상태를 그대로 사용할 수 없다.

파일은 실행 사이에서 자료를 전달한다

저장 장치에 기록한 파일은 다음 실행에서 다시 읽을 수 있다. 파일 저장은 영구 불변을 뜻하지 않으므로 삭제와 덮어쓰기도 구분해야 한다.

파일과 변수를 서로 변환한다

파일의 문자를 읽어 Python 값으로 처리하고, 계산한 결과를 문자열로 바꾸어 저장한다. 오늘은 이 왕복 과정을 작은 텍스트 파일로 확인한다.

코드 예제

```
scores = [80, 90, 100]
average = sum(scores) / len(scores)
print(average)
```

결과 확인

```
90.0
```

print()는 화면 출력이다. 출력했다고 별도의 결과 파일이 자동 생성되지는 않는다.

Files

그림으로 보기: 실행 중인 값과 저장된 파일



메모리와 저장 장치의 역할

그림에서 위쪽은 실행 중의 객체이고 아래쪽은 파일로 남긴 자료이다. 같은 데이터라도 어디에 있는지에 따라 다음 실행에서 다시 읽을 수 있는지가 달라진다.

저장할 시점을 정한다

계산이 끝난 뒤 무엇을 파일에 남길지 정해야 한다. 원본 입력과 출력 결과는 서로 다른 파일로 관리한다.

Text and Binary Files

텍스트 모드와 이진 모드는 해석 방식이 다르다

모든 파일은 저장 장치에서 바이트로 표현된다

텍스트 파일도 실제로는 바이트이다. 텍스트 모드는 지정한 인코딩으로 바이트를 문자로 해석하여 str 객체를 제공한다.

이진 모드는 바이트 자체를 다룬다

rb로 읽은 결과는 bytes이다. 이미지처럼 파일 형식을 그대로 유지해야 하는 자료를 문자로 해석하려고 하지 않는다.

인코딩은 읽기와 쓰기에서 맞춘다

이번 한글 실습 파일은 UTF-8로 저장하고 읽는다. 모든 파일이 UTF-8인 것은 아니며, 확장자만으로 인코딩을 확정할 수 없다.

코드 예제

```
text = "가"
data = text.encode("utf-8")
print(len(text))
print(len(data))
```

결과 확인

```
1
3
```

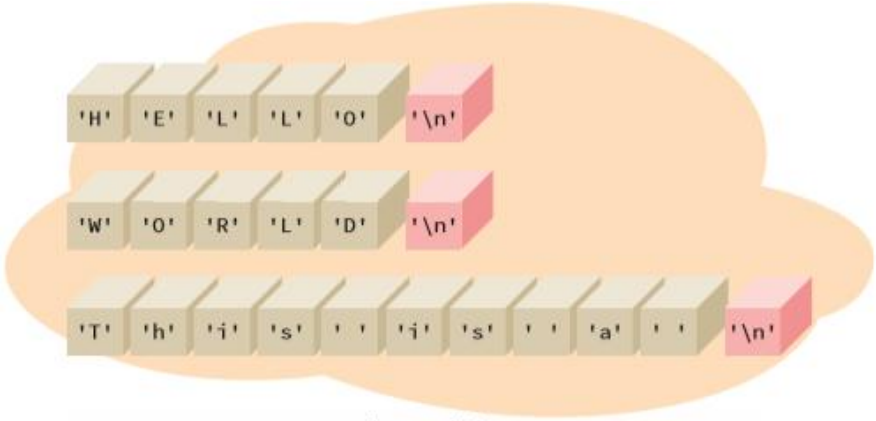
문자 수와 바이트 수는 같은 단위가 아니다.

Text and Binary Files

그림에서 문자와 바이트를 구분하기



텍스트 파일: 문자로 구성된 파일



텍스트 파일

텍스트는 줄과 문자로 읽는다

Hello와 World처럼 사람이 읽는 문자와 줄바꿈을 해석한다. 화면에 보이지 않는 줄바꿈도 문자열의 일부이다.

이미지는 형식에 맞는 프로그램으로 연다

이미지나 압축 파일의 바이트를 임의의 문자로 읽으면 정상 해석되지 않을 수 있다. 복사만 할 때는 바이트를 그대로 옮긴다.

Working Directory

실습 폴더와 파일 위치를 먼저 맞춘다

ch5_lab/

- phones.txt — 실행하는 문서
- hours.txt — 읽을 텍스트 자료
- bridge.png — 복사할 이미지
- scores.txt — 계산할 점수

VS Code에서 ch5_lab 폴더를 열고 그안에 파일들을 저장한다. 자료 파일도 같은 폴더에 준비하며, Notebook의 현재 작업 폴더를 확인한다.

"phones.txt"는 현재 작업 폴더 안의 파일을 찾는다. Notebook 파일이 놓인 위치와 실행 커널의 현재 폴더가 언제나 같다고 가정하지 않는다.

Working Directory

실습 준비: phones.txt를 UTF-8로 저장하기

편집기에서 새 텍스트 파일을 만든다

아래 세 줄을 phones.txt에 저장한다. 한 사람을 한 줄로 작성하고 마지막 줄에도 줄바꿈을 넣는다. 숫자 부분은 계산할 수량이 아니라 전화번호 문자열이다.

경로를 출력하여 위치를 점검한다

Path.cwd()는 현재 작업 폴더를 보여 준다. exists()가 False라면 파일 이름, .txt 확장자, 실제 저장 위치부터 확인한다.

같은 이름의 다른 파일에 주의한다

D 드라이브가 없는 컴퓨터에서도 수업 폴더의 상대 경로를 사용할 수 있다. 경로를 바꿔도 읽을 데이터와 읽기 동작은 그대로이다.

코드 예제

```
from pathlib import Path
print(Path.cwd())
print(Path("phones.txt").exists())
```

Working Directory

Notebook에서 파일 존재와 내용을 확인하기

In [2]:
```
from pathlib import Path
print(Path.cwd().name)
print(Path("phones.txt").exists())
print(Path("phones.txt").read_text(encoding="utf-8"))
```

```
ch5_lab
True
홍길동 010-1234-5678
김철수 010-1234-5679
김영희 010-1234-5680
```

폴더 이름 ch5_lab과 파일 존재 여부 True를 먼저 확인한 뒤 세 줄의 내용을 읽었다.
실행 환경에 따라 전체 경로는 달라지지만 파일 이름과 저장 내용은 같아야 한다.

Opening Files

open()의 모드는 작업 목적을 결정한다

모드	목적	같은 파일이 이미 있으면
r	텍스트 읽기	내용을 지우지 않는다. 없으면 오류가 난다.
w	텍스트 새로 쓰기	열 때 기존 내용을 비운다.
a	텍스트 이어 쓰기	기존 내용 끝에 추가한다.
x	새 텍스트 파일 만들기	이미 있으면 덮어쓰지 않고 오류가 난다.
rb / wb	이진 읽기 / 쓰기	문자 변환 없이 bytes를 다룬다.

파일 객체는 실제 입출력 작업을 제공한다

open("phones.txt", "r", encoding="utf-8")의 반환값은 파일 객체이다. 파일 이름 문자열과 파일 내용 문자열은 서로 다른 값이다.

File Position

읽은 만큼 다음 위치로 이동한다

파일 객체는 다음에 읽을 위치를 기억한다

읽기 모드로 열면 처음에서 시작한다. 같은 객체로 read()를 다시 호출하면 자동으로 처음부터 다시 읽는 것이 아니라 현재 위치부터 읽는다.

EOF는 더 읽을 데이터가 없는 상태이다

텍스트 모드에서 더 읽을 내용이 없으면 빈 문자열을 반환한다. 모든 파일 끝에 특별한 EOF 문자가 실제 저장되어 있다는 뜻은 아니다.

처음부터 읽으려면 위치를 다시 정한다

다시 열거나 seek(0)을 사용한다. 한글 텍스트에서 tell()의 값을 문자 인덱스처럼 임의로 더하고 빼는 방식은 사용하지 않는다.

코드 예제

```
with open("letters.txt",
    encoding="utf-8") as f:
    first = f.read(2)
    rest = f.read()
    empty = f.read()
print(repr(first))
print(repr(empty))
```

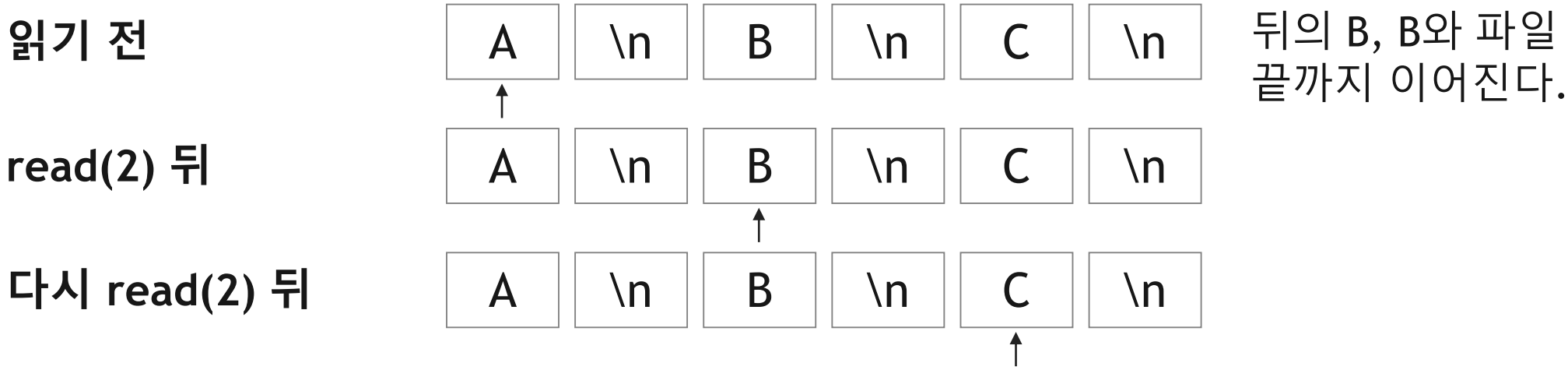
결과 확인

```
'A\n'
''
```

letters.txt에는 A, B, C, B, B를 각각 한 줄씩 저장한다.

File Position

그림으로 보기: 같은 파일 객체의 읽기 위치



뒤의 B, B와 파일 끝까지 이어진다.

read(2)가 "A\n"을 읽은 뒤에는 그 다음 B부터 읽는다. 위치가 파일 끝에 도달한 뒤 read()를 한 번 더 호출하면 빈 문자열이 나온다.

빈 줄도 줄바꿈 문자 한 개를 가진 실제 데이터이다. 파일 끝에서 반환하는 ""와 빈 줄 "\n"을 구분한다.

Reading Files

read(), readline(), readlines() 비교

방법	결과의 자료형	이번 호출에서 읽는 범위
read()	str	현재 위치부터 끝까지
read(n)	str	현재 위치부터 최대 n개 문자
readline()	str	다음 한 줄
readlines()	list[str]	남은 줄 전체를 리스트로
for line in f	매 반복의 line은 str	남은 줄을 차례로 처리

비교 실험은 같은 시작 조건에서 한다

한 객체로 순서대로 모두 실행하면 앞선 읽기가 위치를 바꾸므로 같은 내용을 비교할 수 없다. 각 방법을 비교할 때는 파일을 다시 연다.

Reading Files

read()의 결과는 하나의 긴 문자열이다

In [4]:
```
with open("phones.txt", encoding="utf-8") as infile:
    text = infile.read()
print(type(text))
print(text, end="")
```

```
<class 'str'>
홍길동 010-1234-5678
김철수 010-1234-5679
김영희 010-1234-5680
```

In [5]:
```
print(repr(text[:20]))
```

```
'홍길동 010-1234-5678\n김철'
```

화면에는 세 줄로 보이지만 text는 str 하나이다. repr()는 문자열 일부에 포함된 줄바꿈을 \n으로 보여 주므로 보이지 않는 문자를 확인하는 데 도움이 된다.

Reading Files

readline()으로 빈 줄과 파일 끝 구분하기

In [9]:
```
with open("blank.txt", encoding="utf-8") as f:
    for i in range(4):
        print(i + 1, repr(f.readline()))
```

```
1 'A\n'
2 '\n'
3 'B\n'
4 ''
```

blank.txt에 A, 빈 줄, B를 차례로 저장한 예이다. 두 번째 결과는 줄바꿈이 들어 있는 문자열이고, 네 번째 결과는 더 이상 읽을 줄이 없음을 나타내는 빈 문자열이다.

Reading Files

readlines()의 결과는 줄 문자열들의 리스트이다

In [11]:
```
with open("phones.txt", encoding="utf-8") as infile:
    lines = infile.readlines()
print(type(lines))
print(len(lines))
print(repr(lines[0]))
```

```
<class 'list'>
3
'홍길동 010-1234-5678\n'
```

In [12]:
```
for line in lines:
    print(line, end="")
```

```
홍길동 010-1234-5678
김철수 010-1234-5679
김영희 010-1234-5680
```

lines의 길이 3은 전화번호 항목이 세 줄이라는 뜻이다. 첫 원소에도 줄바꿈이 남아 있다. 리스트 자체를 출력하는 것과 각 줄을 출력하는 것을 구분한다.

Reading Lines

파일 반복문에서 줄바꿈이 두 번 생기는 이유

읽은 줄에 이미 줄바꿈이 들어 있다

파일의 한 줄을 읽으면 보통 끝의 \n도 함께 읽는다. print(line)은 이 줄바꿈 뒤에 print()의 기본 줄바꿈을 다시 붙여 빈 줄처럼 보이게 할 수 있다.

원래 줄바꿈을 그대로 출력한다

print(line, end="")는 추가 줄바꿈을 붙이지 않는다. 파일에 저장된 줄 모양을 그대로 보고 싶을 때 적합하다.

제거 범위를 목적에 맞게 정한다

strip()은 양끝 공백도 지운다. 줄바꿈만 없애려면 rstrip("\n")을 사용한다. 이름이나 문장 안의 의미 있는 공백을 무조건 지우지 않는다.

코드 예제

```
with open("phones.txt",
    encoding="utf-8") as f:
    for line in f:
        print(line, end="")
```

결과 확인

```
홍길동 010-1234-5678
김철수 010-1234-5679
김영희 010-1234-5680
```

Reading Lines

같은 문자열에서 strip()의 범위를 비교하기

In [14]:
```
line = "  Alice  \n"
print(repr(line))
print(repr(line.strip()))
print(repr(line.rstrip()))
print(repr(line.rstrip("\n")))
```

```
'  Alice  \n'
'Alice'
'  Alice'
'  Alice  '
```

첫 줄은 원래 문자열, 다음은 strip(), rstrip(), rstrip("\n")의 결과이다. 파일 내용의 앞뒤 공백도 데이터일 수 있으므로 줄바꿈 제거와 공백 정리를 같은 작업으로 보지 않는다.

Closing Files

파일은 사용이 끝나면 닫는다

open()과 close()를 한 쌍으로 이해한다

파일 사용이 끝나면 close()로 관련 자원을 해제한다. 닫힌 파일 객체로 다시 읽거나 쓰려고 하면 오류가 발생하므로 새로 열어야 한다.

with는 사용 범위를 코드로 드러낸다

with open(...) as f: 아래 들여쓴 구간에서 파일을 사용한다. 이 구간을 벗어날 때 파일을 닫으므로 close()를 빠뜨리는 일을 줄인다.

종료와 저장 완료를 구분한다

쓰기 데이터는 버퍼에 남을 수 있다. 파일 닫기는 정리와 출력 반영을 수행하지만, 모든 전원 장애 상황의 디스크 영구 저장까지 보증하는 것은 아니다.

코드 예제

```
infile = open("phones.txt",
    encoding="utf-8")
text = infile.read()
infile.close()

with open("phones.txt",
    encoding="utf-8") as f:
    text = f.read()
```

두 방식은 같은 내용을 읽는다. 이후 실습에서는 with 구문을 기본으로 사용한다.

Closing Files

with 구간 안과 밖의 closed 상태

In [16]:
```
with open("letters.txt", encoding="utf-8") as f:
    print(f.closed)
    print(f.read())
print(f.closed)
```

```
False
A
B
C
B
B

True
```

In [17]:
```
f.read()
```

```
ValueError: I/O operation on closed file.
```

with 안에서는 False, 구간을 벗어난 뒤에는 True이다. 마지막 호출은 닫힌 파일에서 read()를 실행한 의도적 오류이며, 화면에는 예외 종류와 메시지를 표시했다.

Practice

실습 1: 전화번호 파일에 줄 번호 붙이기

완성할 동작

phones.txt를 한 줄씩 읽어 1부터 줄 번호를 붙여 출력한다. 마지막에는 읽은 사람 수를 출력한다. 원본 파일은 수정하지 않는다.

먼저 예상할 내용

빈 줄이 없는 세 줄 파일이므로 마지막 count는 3이다. 줄 번호와 전화번호 자체의 숫자는 서로 다른 정보이다.

두 곳의 빈칸을 채운다

반복마다 count를 갱신하고, print()에서 줄바꿈이 중복되지 않도록 처리한다. 빈 줄이 있는 파일은 별도 조건을 추가해야 한다.

실습 코드

```
count = 0
with open("phones.txt",
    encoding="utf-8") as f:
    for line in f:
        count = _____________
        print(count, _____________)
print("인원:", count)
```

실행하기 전에 첫 줄과 마지막 줄의 출력을 적어 본다.

Practice

해설: 한 줄을 읽을 때마다 개수를 증가시킨다

반복 전에 count를 0으로 둔다

아직 읽은 줄이 없으므로 0에서 시작한다. 한 줄을 받으면 1을 더한 뒤 출력하므로 첫 줄 번호는 1이다.

strip의 대상은 읽은 문자열이다

rstrip("\n")은 line에서 줄바꿈을 제거한 새 문자열을 만든다. 파일 안의 내용을 직접 지우는 명령이 아니다.

확인할 변형

네 번째 사람을 파일에 추가하고 다시 실행하면 count는 4가 된다. 커널의 과거 count가 남지 않도록 초기화부터 실행한다.

실습 코드

```
count = 0
with open("phones.txt",
    encoding="utf-8") as infile:
    for line in infile:
        count += 1
        print(count, line.rstrip("\n"))
print("인원:", count)
```

이번 파일에는 빈 줄이 없다고 가정한다. 모든 줄의 수와 실제 인원수가 항상 같은 것은 아니다.

Practice

전화번호 파일의 실제 출력 확인

In [19]:
```
count = 0
with open("phones.txt", encoding="utf-8") as infile:
    for line in infile:
        count += 1
        print(count, line.rstrip("\n"))
print("인원:", count)
```

```
1 홍길동 010-1234-5678
2 김철수 010-1234-5679
3 김영희 010-1234-5680
인원: 3
```

변수 count와 파일 위치가 반복마다 함께 바뀐다. 첫 줄은 1, 세 번째 줄은 3이며, 반복 종료 뒤의 인원 출력은 줄바꿈을 제거하는 처리와 별개이다.

Parsing Text

한 줄의 문자열을 여러 값으로 나누기

hours.txt의 한 줄은 여러 항목을 담는다

첫 항목은 식별 번호, 두 번째는 이름, 나머지는 시간 값이다. 파일을 읽기만 하면 모든 항목은 아직 문자열에 들어 있다.

split()으로 항목을 구분한다

공백으로 나눈 결과는 문자열 리스트이다. fields[1]로 이름을 읽고 fields[2:]로 시간 항목들을 선택한다.

수치 계산 전에 float()로 변환한다

12.5처럼 소수를 처리하므로 float()를 사용한다. 이름과 식별 번호까지 숫자로 바꾸려 하지 않는다.

코드 예제

```
line = "123 Susan 12.5 8.1 7.6 3.2"
fields = line.split()
print(fields[1])
print(fields[2:])
```

결과 확인

```
Susan
['12.5', '8.1', '7.6', '3.2']
```

hours.txt에는 다음 화면에서 제시하는 제시한 세 줄을 사용한다.

Practice

실습 2: 사람별 시간 합계 계산하기

123 Susan 12.5 8.1 7.6 3.2
456 Brad 4.0 11.6 6.5 2.7 12
789 Jenn 8.0 8.0 8.0 8.0 7.5

hours.txt로 저장할 내용

위 세 줄은 Python 코드가 아니라 텍스트 파일 내용이다. 각 이름 뒤에 있는 시간 값의 개수는 서로 다를 수 있다.

요구사항

한 줄씩 split()한 뒤 이름을 출력한다. 시간 항목을 float()로 바꾸어 모두 더하고 합계를 소수점 한 자리로 출력한다.

예상 결과

Susan의 합계는 31.4이다. Brad와 Jenn의 합계도 직접 예상하고, 결과를 확인한다. total은 사람마다 0으로 초기화한다.

Practice

시간 항목만 변환하여 더한 실행 결과

In [21]:
```
with open("hours.txt", encoding="utf-8") as infile:
    for line in infile:
        fields = line.split()
        name = fields[1]
        total = 0.0
        for text in fields[2:]:
            total += float(text)
        print(name, f"{total:.1f}")
```

```
Susan 31.4
Brad 36.8
Jenn 39.5
```

Susan 31.4, Brad 36.8, Jenn 39.5가 출력된다. 바깥 반복은 사람을 바꾸고, 안쪽 반복은 그 사람의 시간 항목을 더한다. 입력 파일의 내용은 변하지 않는다.

Writing Files

w 모드는 기존 내용을 먼저 비운다

새 파일과 기존 파일의 동작이 다르다

파일이 없으면 새로 만든다. 같은 경로의 파일이 있으면 open(..., "w") 시점에 기존 내용을 비운다. write()를 아직 하지 않았어도 지워질 수 있다.

화면 출력과 파일 기록은 별개이다

print()로 본 계산 결과를 파일에 남기려면 write()가 필요하다. 파일을 쓴 뒤에는 닫고 다시 읽어 내용을 확인한다.

실습 파일 이름을 별도로 정한다

이번에는 memo.txt, phones1.txt처럼 출력 목적의 파일만 새로 만든다. 수업자료나 개인 문서를 대상으로 w 모드를 시험하지 않는다.

코드 예제

```
with open("memo.txt",
    "w", encoding="utf-8") as f:
    f.write("OLD\n")

with open("memo.txt",
    "w", encoding="utf-8") as f:
    pass
```

두 번째 with가 끝난 뒤 memo.txt의 내용은 빈 문자열이다.

Writing Files

write()를 하지 않아도 기존 내용이 사라지는 경우

In [23]:
```
with open("memo.txt", "w", encoding="utf-8") as f:
    f.write("OLD\n")
with open("memo.txt", encoding="utf-8") as f:
    print("처음:", repr(f.read()))
```

```
처음: 'OLD\n'
```

In [24]:
```
with open("memo.txt", "w", encoding="utf-8") as f:
    pass
with open("memo.txt", encoding="utf-8") as f:
    print("w로 다시 연 뒤:", repr(f.read()))
```

```
w로 다시 연 뒤: ''
```

첫 파일에는 OLD와 줄바꿈이 있다. 두 번째에는 w로 열고 아무것도 쓰지 않았는데도 내용이 빈 문자열이 된다. 덮어쓰기 위험은 write() 호출 직전이 아니라 파일을 열기 전에 점검한다.

Writing Modes

이어 쓰기와 새 파일 전용 모드 비교

상황	선택	주의할 점
이전 결과를 새 결과로 교체	w	이전 내용이 사라져도 되는지 확인
실행 기록을 뒤에 누적	a	재실행하면 같은 줄도 중복 추가
이미 있는 결과를 보호	x	파일이 있으면 FileExistsError
기존 파일을 읽기만 함	r	읽기용 파일을 실수로 w로 열지 않음

같은 코드를 두 번 실행하는 상황도 생각한다

Notebook은 셀을 반복 실행하기 쉽다. a는 기록을 누적하고 w는 매번 바꾸며 x는 두 번째 생성에 실패한다. 어떤 동작이 필요한지 먼저 정한다.

Writing Text

write()는 문자열을 쓰고 줄바꿈을 자동 추가하지 않는다

문자열 형식으로 바꾸어 전달한다

텍스트 파일의 write()는 str을 받는다. 숫자를 저장할 때는 str(value)나 f-string으로 출력 문자열을 만든다.

한 줄 끝은 직접 표시한다

f.write("Hello") 다음에 f.write("Python")을 실행하면 두 내용이 붙는다. 다음 줄에 쓰려면 문자열에 \n을 포함한다.

반환값은 파일 내용이 아니다

write()의 반환값은 쓴 문자 수이다. 화면에 숫자가 자동 표시되어도 그 숫자를 파일에 기록했다는 뜻은 아니다.

코드 예제

```
with open("output.txt",
    "w", encoding="utf-8") as f:
    f.write("Hello, world!\n")
    f.write("How are you?")
```

결과 확인

Hello, world!
How are you?

텍스트 모드의 줄바꿈은 운영체제에 따라 저장 바이트가 달라질 수 있다.

Writing Text

같은 두 단어를 붙여 쓰거나 줄을 나누어 쓰기

In [28]:
```
with open("joined.txt", "w", encoding="utf-8") as f:
    n = f.write("Hello")
    f.write("Python")
print("첫 write의 반환값:", n)
with open("joined.txt", encoding="utf-8") as f:
    print(repr(f.read()))
```

```
첫 write의 반환값: 5
'HelloPython'
```

In [29]:
```
with open("lines.txt", "w", encoding="utf-8") as f:
    f.write("Hello\nPython\n")
with open("lines.txt", encoding="utf-8") as f:
    print(repr(f.read()))
```

```
'Hello\nPython\n'
```

첫 파일은 HelloPython이라는 한 줄이고 첫 write()의 반환값은 5이다. 두 번째 파일은 Hello와 Python 사이와 끝에 줄바꿈이 있다. repr()로 파일 내용을 비교했다.

Practice

실습 3: 전화번호 파일을 코드로 생성하기

이번에는 편집기 대신 Python으로 기록한다

phones1.txt라는 별도 파일에 세 사람을 한 줄씩 저장한다. 파일 이름이 기존 입력 phones.txt와 다름을 확인한다.

읽기와 쓰기 인코딩을 동일하게 지정한다

한글을 UTF-8로 저장하고 다시 UTF-8로 읽는다. 쓰기에서 인코딩을 생략한 뒤 읽기에서만 지정하지 않는다.

출력 확인까지 완료해야 한다

셀 실행 성공만 확인하지 말고 phones1.txt를 편집기로 열거나 다시 읽는다. 세 줄과 전화번호의 하이픈이 유지되어야 한다.

실습 코드

```
with open("phones1.txt", "w",
          encoding="utf-8") as out:
    out.write("홍길동 010-1234-5678\n")
    out.write("김철수 010-1234-5679\n")
    out.write("김영희 010-1234-5680\n")
```

세 개의 write() 호출은 각각 한 줄을 기록한다.

Practice

직접 만든 전화번호 파일 다시 읽기

In [31]:
```
with open("phones1.txt", "w", encoding="utf-8") as out:
    out.write("홍길동 010-1234-5678\n")
    out.write("김철수 010-1234-5679\n")
    out.write("김영희 010-1234-5680\n")
with open("phones1.txt", encoding="utf-8") as f:
    print(f.read(), end="")
```

```
홍길동 010-1234-5678
김철수 010-1234-5679
김영희 010-1234-5680
```

작성 구간을 벗어나 파일을 닫은 후 다시 읽었다. 읽기 결과가 원래 의도한 세 줄과 같아야 한다. 화면에 출력된 자료는 phones1.txt에 기록한 내용이다.

Practice

실습 4: 텍스트 파일의 내용 복사하기

입력과 출력 파일 이름을 다르게 지정한다

phones.txt의 내용을 phones_copy.txt로 옮긴다. 같은 파일을 입력과 출력으로 지정하면 출력 파일을 열 때 원본이 비워질 위험이 있다.

읽고 쓰는 작업을 순서대로 수행한다

입력 파일을 읽어 문자열을 얻은 뒤 그 문자열을 출력 파일에 기록한다. 작은 텍스트 파일이므로 전체 내용을 한 번에 읽는다.

복사한 결과를 다시 읽어 비교한다

내용이 같다는 판단은 파일명 두 개가 존재한다는 사실만으로 끝나지 않는다. 이번에는 같은 인코딩으로 읽은 문자열의 일치를 확인한다.

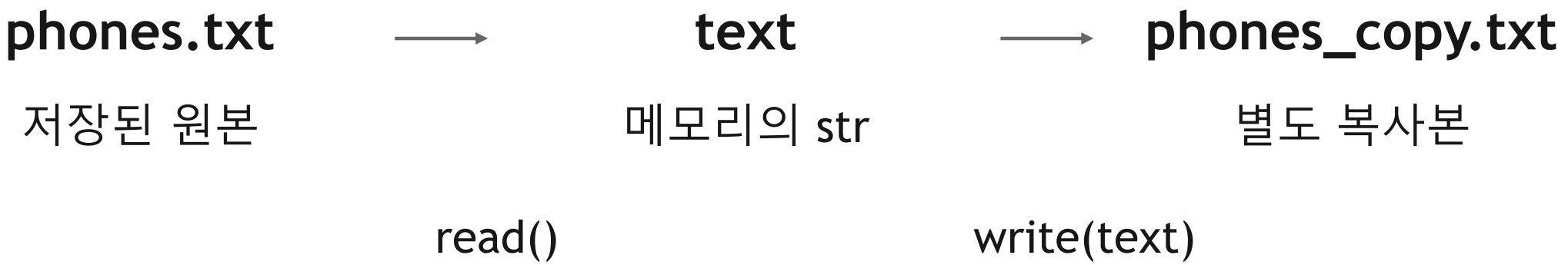
실습 코드

```
with open("phones.txt",
    encoding="utf-8") as src:
    text = src.read()
with open("phones_copy.txt", "x",
          encoding="utf-8") as dst:
    dst.write(text)
```

x는 이미 있는 복사본을 보호한다. 재실행할 때는 새 출력 이름을 사용한다.

Copying Files

원본을 읽고 다른 파일에 쓰기



phones.txt에서 읽은 text는 Python 메모리의 문자열이다. text를 phones_copy.txt에 쓰는 순간 두 번째 파일에 내용이 기록된다.

같은 문자열을 읽고 썼어도 인코딩과 줄바꿈 변환에 따라 저장 바이트까지 같다는 보장은 없다. 이미지 파일에서는 이진 복사가 필요하다.

Copying Files

복사본을 읽어 원래 문자열과 비교하기

In [33]:
```
with open("phones.txt", encoding="utf-8") as src:
    text = src.read()
with open("phones_copy.txt", "x", encoding="utf-8") as dst:
    dst.write(text)
with open("phones_copy.txt", encoding="utf-8") as f:
    copied = f.read()
print(text == copied)
print(copied, end="")
```

```
True
홍길동 010-1234-5678
김철수 010-1234-5679
김영희 010-1234-5680
```

True는 원본과 복사본에서 읽은 문자열이 같다는 비교 결과이다. 입력 파일을 변경하지 않고 별도의 복사본을 만들었다. 이 확인은 모든 파일 메타정보의 일치 검사는 아니다.

Binary Files

한글 한 글자를 문자와 바이트로 읽기

In [35]:
```
with open("korean.txt", encoding="utf-8") as f:
    text = f.read()
with open("korean.txt", "rb") as f:
    data = f.read()
print(text, type(text), len(text))
print(data, type(data), len(data))
```

```
가 <class 'str'> 1
b'\xea\xb0\x80' <class 'bytes'> 3
```

korean.txt에는 가 한 글자를 줄바꿈 없이 UTF-8로 저장했다. 텍스트 모드에서는 길이 1인 str이고 이진 모드에서는 길이 3인 bytes이다. 길이를 해석할 때 단위를 먼저 확인한다.

Binary Copy

1024바이트씩 읽어 이미지 파일 복사하기

```
with open("bridge.png", "rb") as src:
    with open("bridge_copy.png", "xb") as dst:
        while True:
            block = src.read(1024)
            if not block:
                break
            dst.write(block)
```

Binary Copy

동작 해설: 1024바이트씩 읽어 이미지 파일 복사하기

이미지 파일은 bytes로 복사한다

rb와 xb는 인코딩을 지정하지 않는다. 각 block을 그대로 쓰므로 텍스트의 문자 변환이나 줄바꿈 변환이 없다.

마지막 조각은 더 작을 수 있다

read(1024)는 최대 1024바이트를 읽는다. 남은 바이트가 더 적으면 그만큼만 받고, 더 읽을 내용이 없을 때 b""를 반환한다.

원본과 출력 경로를 분리한다

bridge.png를 수업 폴더에 준비한다. bridge_copy.png가 이미 있으면 새 이름을 사용하며, 이 예제는 기존 출력 덮어쓰기를 허용하지 않는다.

Binary Copy

그림으로 보기: 2500바이트를 나누어 읽기



1024 + 1024 + 452 = 2500 bytes

다음 read(1024) → b"" → 종료

2500바이트는 1024, 1024, 452바이트로 나뉜다. 다음 읽기가 0바이트를 반환하면 루프를 종료하므로 빈 조각을 기록하지 않는다.

read(1024)가 있다고 반복이 1024번 실행되는 것은 아니다. 마지막 조각의 길이와 종료 조건을 연결하여 설명한다.

Binary Copy

읽어 온 조각의 길이를 실제로 확인하기

In [37]:
```
sizes = []
with open("sample.bin", "rb") as f:
    while True:
        block = f.read(1024)
        sizes.append(len(block))
        if not block:
            break
print(sizes)
print(sum(sizes))
```

```
[1024, 1024, 452, 0]
2500
```

sample.bin은 설명을 위해 준비한 2500바이트 파일이다. 리스트 마지막의 0은 추가 데이터가 없다는 종료 확인이며, 세 조각 길이의 합계는 2500이다.

Binary Copy

복사한 이미지가 같은 모습인지 확인하기


->
->
->

Binary Copy

실제 이미지 파일의 바이트 일치 확인

In [2]:
```
with open("bridge.png", "rb") as f:
    original = f.read()
with open("bridge_copy.png", "rb") as f:
    copied = f.read()
print("바이트 일치:", original == copied)
print("원본 크기:", len(original))
print("복사본 크기:", len(copied))
```

```
바이트 일치: True
원본 크기: 70404
복사본 크기: 70404
```

bridge.png를 읽어 다른 이름으로 복사하고 두 파일의 bytes를 비교했다. 출력의 True는 이 실습에서 복사한 파일 내용이 일치함을 뜻한다.

Practice

종합 실습: 점수 파일에서 평균을 계산하고 저장하기

입력 파일을 먼저 준비한다

scores.txt에 80, 90, 100을 각각 한 줄로 저장한다. 이번 실습은 빈 줄이 없고 모든 줄이 정수이며 최소 한 줄이 있다고 가정한다.

읽기와 계산을 연결한다

한 줄씩 int()로 바꾸어 합계와 개수를 누적한다. 반복이 끝난 뒤 평균을 계산하고 summary.txt에 개수와 평균을 기록한다.

파일이 둘 다 남아 있어야 한다

scores.txt는 그대로 유지하고 summary.txt만 새 결과로 쓴다. 평균이 90.0인지 파일을 다시 읽어 확인한다.

실습 코드

```
total = 0
count = 0
with open("scores.txt",
    encoding="utf-8") as f:
    for line in f:
        total += _____________
        count += 1
average = _____________
```

마지막 저장에는 f-string과 줄바꿈을 사용한다. 잘못된 입력의 처리는 다음 차시에서 배운다.

Practice

해설: 입력 파일과 결과 파일의 역할을 나눈다

```
total = 0
count = 0
with open("scores.txt", encoding="utf-8") as f:
    for line in f:
        total += int(line)
        count += 1
average = total / count
with open("summary.txt", "w", encoding="utf-8") as f:
    f.write(f"count={count}\naverage={average:.1f}\n")
print(count, average)
```

Practice

동작 해설: 입력 파일과 결과 파일의 역할을 나눈다

누적은 읽는 반복문 안에서

int(line)으로 각 줄을 변환한다. total과 count는 반복이 시작되기 전에 초기화하고, average는 모든 값을 읽은 뒤에 계산한다.

평균을 문자열로 저장한다

f-string으로 만든 두 줄을 write()에 전달한다. 개수는 3, 평균은 90.0이며 summary.txt에 같은 내용이 남는다.

이번 해설의 전제

빈 파일이면 count가 0이므로 나눗셈을 그대로 실행할 수 없다. 정상 입력과 오류 입력의 처리를 나누어 생각한다.

Practice

평균 계산과 summary.txt의 내용

```
[41] 3 90.0

[42] count=3
     average=90.0
```

첫 출력은 계산한 개수와 평균이다. 다음 출력은 저장한 summary.txt를 다시 읽은 내용이다. 화면 출력과 파일 저장이 별도로 수행되었음을 확인한다.

Chapter Review

마지막 확인: 어떤 동작을 선택해야 하는가?

질문	생각할 기준
파일의 다음 한 줄만 읽으려면?	readline()과 readlines()의 반환값
read()를 두 번 실행했는데 두 번째가 비었다면?	현재 읽기 위치와 EOF
같은 파일에 기록을 이어 붙이려면?	a의 재실행 시 누적 동작
이미지 복사에 UTF-8을 지정해야 하는가?	str과 bytes의 차이
내용을 다 읽은 뒤 다시 처음부터 읽으려면?	다시 열기 또는 seek(0)

답은 실행 이유와 함께 말한다

메서드 이름만 외우기보다 입력 자료형, 읽는 위치, 원본 변경 여부를 기준으로 설명한다.
다음 차시에는 파일이 없거나 입력이 잘못된 경우까지 처리한다.

Chapter Overview

읽기 위치, 자료형, 저장 모드를 함께 확인한다

읽기에서 기억할 내용

readline()은 다음 한 줄, readlines()는 남은 줄 목록이다. read() 이후 빈 문자열이 나왔다면 현재 위치가 끝인지 먼저 확인한다. 처음으로 돌아갈 때는 다시 열거나 seek(0)을 사용한다.

쓰기에서 기억할 내용

a는 뒤에 추가하고 w는 열 때 기존 내용을 비운다. write()는 줄바꿈을 자동으로 넣지 않는다. with 구간이 끝난 뒤 결과 파일을 다시 읽어 확인한다.

다음 차시의 질문

파일이 없거나 점수가 숫자가 아니어도 프로그램을 계속 사용하려면 어떻게 해야 할까? 예외를 구분하고 필요한 부분만 처리한 뒤, 코드를 모듈로 나누어 재사용한다.